Introduction to Computer Science: Homework 3

Yifan Chen

Fall 2026

Problem 1 — Finite Precision (3 points)

A computer can store only finitely many bits, so it cannot represent every real number exactly. Consider a simple number system that stores a number using exactly three binary digits:

$$(0.b_1b_2b_3)_2, \qquad b_i \in \{0, 1\}.$$

The value of such a binary number is

$$(0.b_1b_2b_3)_2 = \frac{b_1}{2} + \frac{b_2}{4} + \frac{b_3}{8}.$$

For example,

$$(0.101)_2 = \frac{1}{2} + \frac{1}{8} = 0.625.$$

When a real number is first stored in this system, it is rounded to the nearest representable value. You may assume that no input considered in this problem lies exactly halfway between two representable values.

Arithmetic operations are first evaluated exactly. Before the result is stored again, however, only the three binary fractional digits $b_1b_2b_3$ are retained. All other bits are discarded, including both any integer bits and any fractional bits beyond the third. For example,

$$(1.101101)_2 \to (0.101)_2.$$

(a) List all numbers between 0 and 1 that can be represented exactly in this system.

(b) Can the decimal number 0.3 be represented exactly? If not, find the representable number closest to 0.3.

(c) Is it always true that

$$(a + b) + c = a + (b + c)$$

in this number system? Either prove that it is always true or give a counterexample.

(d) (Optional, 0 points) Is it always true that

$$(a \times b) \times c = a \times (b \times c)$$

in this number system? Either prove that it is always true or give a counterexample.

Answer:

(a) The numbers are $0, \frac{1}{8}, \frac{1}{4}, \frac{3}{8}, \frac{1}{2}, \frac{5}{8}, \frac{3}{4}, \frac{7}{8}, 1$.

(b) 0.3 cannot be represented exactly, and the closest representable number is $\frac{1}{4}$, which can be represented as $(0.010)_2$.

(c) The associative property of addition is always true, since the numbers will first be changed into the closest representable number. As we only consider the decimal part, the sum of the representable numbers is still representable, then the addition is the same as the addition on $\mathbb{R}$, which has the associative property.

(d) The associative property of multiplication is always true, since the numbers will first be changed into the closest representable number, which can be written in a polynomial with $x = 2^{-1}$ and degree less than 4. Then product of the representable numbers can be seen as a multiple of polynomials under $(\text{mod } x^4)$, which has the associative property.

Problem 2 — Instruction-Level Parallelism (3 points)

In this week's lecture, we discussed instruction-level parallelism as one way to improve processor performance. Modern processors may execute multiple independent instructions at the same time, but data dependencies between instructions can limit the amount of parallelism that can actually be exploited.

Consider the following sequence of instructions:

$$\begin{aligned} I_1: &\quad a = b + c, \\ I_2: &\quad d = a \times e, \\ I_3: &\quad f = d + g, \\ I_4: &\quad h = i + j, \\ I_5: &\quad k = l - m, \\ I_6: &\quad n = h \times k, \\ I_7: &\quad p = f + n. \end{aligned}$$

Assume that all input values b, c, e, g, i, j, l, m are available before execution begins. For simplicity, we use the following processor model throughout this problem.

- Each instruction takes exactly one cycle to execute.
- The result of an instruction executed in cycle t becomes available at the beginning of cycle $t + 1$.
- An instruction may execute only if all values it requires are available at the beginning of that cycle.
- Ignore instruction fetch, decoding, memory accesses, and all dependencies other than those caused by values produced by earlier instructions.

Answer:

(a) An instruction has a *data dependency* on an earlier instruction if it uses a value produced by that instruction. For each instruction $I_2, \ldots, I_7$, fill in its direct data dependency or dependencies in the table below. Write "None" if the instruction has no such dependency.

Instruction	Direct data dependency/dependencies
I_2	I_1
I_3	I_2
I_4	None
I_5	None
I_6	I_4, I_5
I_7	I_3, I_6

(b) First consider a processor that can execute at most two instructions in one cycle, but must execute instructions in program order. The processor always considers the earliest instruction that has not yet executed. If this instruction cannot execute because one of its input values is unavailable, no later instruction may execute in that cycle. Otherwise, the processor may also execute the next instruction in program order in the same cycle, provided that its inputs are also available at the beginning of the cycle.

Fill in the execution schedule below. Leave unused rows blank. How many cycles are required to execute all seven instructions?

Cycle	Instruction 1	Instruction 2
1	I_1	
2	I_2	
3	I_3	I_4
4	I_5	
5	I_6	
6	I_7	
7		

(c) Now consider a processor that can still execute at most two instructions in one cycle, but may execute instructions out of program order. In each cycle, it may choose any two instructions whose input values are available, as long as all data dependencies are respected.

Fill in a schedule that completes all seven instructions in the minimum possible number of cycles. Leave unused rows blank. What speedup does this achieve compared with a processor that executes exactly one instruction per cycle?

Cycle	Instruction 1	Instruction 2
1	I_1	I_4
2	I_2	I_5
3	I_3	I_6
4	I_7	
5		
6		
7		

The speedup is $7/4 = 175\%$.

(d) Finally, suppose the out-of-order processor is widened so that it can execute at most four instructions in one cycle. What is the minimum possible number of cycles required now? Justify why the program cannot be completed in fewer cycles, even if the processor were able to execute an arbitrary number of instructions in parallel.

The answer is still 4 since there is a dependency chain $I_1 \to I_2 \to I_3 \to I_7$ and each cycle can compute at most one of them.

Problem 3 — Cache Memory (4 points)

In this week's lecture, we discussed the memory wall: processors can perform computations much faster than data can be transferred from main memory. Modern computer systems therefore use a small and fast memory called a *cache* between the processor and main memory.

When the processor requests some data, it first checks whether the data is already stored in the cache. If so, the request is a *cache hit*. Otherwise, it is a *cache miss*, and the data must be fetched from main memory and placed into the cache.

In this problem, we will study several simple cache designs.

3.1 A direct-mapped cache

Main memory is divided into blocks, numbered $0, 1, 2, 3, \ldots$, and suppose the cache contains four cache lines, numbered $0, 1, 2, 3$. Each cache line can store exactly one memory block.

We first consider a direct-mapped cache. Memory block b can only be stored in cache line $b \bmod 4$. For example, blocks 0, 4, and 8 must all use cache line 0.

When block b is requested:

- if cache line $b \bmod 4$ already contains block b, the request is a cache hit;
- otherwise, the request is a cache miss, and block b replaces whatever block was previously stored in that cache line.

Assume that the cache is initially empty.

Answer:

(a) Consider the following sequence of memory-block requests:

$$0, 1, 2, 3, 0, 4, 0, 1, 5, 1.$$

Complete the table below. In the "Hit/Miss" column, write H for a cache hit and M for a cache miss. In the remaining columns, write the block stored in each cache line after the request has been processed. Use "–" for an empty cache line.

Step	Requested block	Hit/Miss	Line 0	Line 1	Line 2	Line 3
1	0	M	0	–	–	–
2	1	M	0	1	–	–
3	2	M	0	1	2	–
4	3	M	0	1	2	3
5	0	H	0	1	2	3
6	4	M	4	1	2	3
7	0	M	0	1	2	3
8	1	H	0	1	2	3
9	5	M	0	5	2	3
10	1	M	0	1	2	3

What is the cache hit rate for this sequence?

The hit rate is $2/10 = 20\%$.

3.2 Why store a block instead of one value?

So far, each memory block has represented one piece of data. Real caches usually fetch several nearby memory locations together. This takes advantage of the fact that nearby data are often accessed close together in time.

Suppose memory is now divided into individual words, numbered $0, 1, 2, \ldots$. Consider two cache designs, both capable of storing a total of 16 words:

- Cache A: each block contains one word, so the cache has 16 blocks;
- Cache B: each block contains four consecutive words, so the cache has 4 blocks. Block b contains words $4b, 4b + 1, 4b + 2, 4b + 3$.

Assume that both caches are initially empty, and that none of the blocks loaded below are replaced before the end of the sequence.

(b) The processor requests the following words:

$$0, 1, 2, 3, 8, 9, 10, 11.$$

For Cache A and Cache B separately, determine which requests are hits and which are misses, and compute the hit rate.

Explain briefly why the two caches behave differently even though they can store the same total number of words.

Answer: For Cache A, all the requests are misses, so the hit rate is $0/8 = 0\%$. For Cache B, only request 0 and 8 are misses, so the hit rate is $6/8 = 75\%$.

The difference is caused by the locality of accesses, one block tends to be accessed in continuous cycles, so Cache B has only two cold misses.

3.3 When blocks fight for the same place

Direct mapping is simple, but different memory blocks may be forced to use the same cache line.

Return to memory blocks numbered $0, 1, 2, \ldots$ and a cache that can store four blocks in total. Compare the following two cache designs:

- Direct-mapped: four cache lines, where block b must be placed in line $b \bmod 4$.
- 2-way set-associative: the cache is divided into two sets, each of which can store two blocks. Block b must be placed in set $b \bmod 2$, but it may occupy either of the two locations in that set.

If both locations are occupied when a new block must be inserted, the block that has not been requested for the longest time is replaced. This rule is called least recently used (LRU) replacement.

Again, assume that both caches are initially empty.

(c) Consider the request sequence

$$0, 4, 0, 4, 0, 4.$$

Determine the hit/miss sequence and hit rate for both cache designs.

Both caches can store exactly four blocks. Why can their hit rates still be very different for the same request sequence?

Answer: The hit rate of direct-mapped is 0% while the hit rate of 2-way set-associative is $4/6 = 66.6\%$, since the blocks need the same cache line, so they alternately replace each other in direct-mapped, while set-associative allows the blocks to be saved simultaneously.

3.4 Is a lower miss rate always better?

A more sophisticated cache may reduce the miss rate, but it may also take longer to check.

Consider two cache designs:

	Hit time	Miss rate
Cache X	1 ns	10%
Cache Y	2 ns	5%

If a request misses in the cache, accessing main memory requires an additional P ns. Thus, the average memory-access time is

$$T_{\text{avg}} = T_{\text{hit}} + (\text{miss rate})P.$$

(d) Suppose first that $P = 50$ ns. Compute the average memory-access time of Cache X and Cache Y. Now treat P as an unknown positive number. For what values of P is Cache Y faster than Cache X?

Based on your result, explain briefly why a cache with a lower miss rate is not necessarily a faster cache.

Answer: For $P = 50$ ns, we have $T_{\text{avg}} = 1\text{ ns} + 10\% \cdot 50\text{ ns} = 6$ ns for X and $T_{\text{avg}} = 2\text{ ns} + 5\% \cdot 50\text{ ns} = 4.5$ ns for Y. For Y to be faster than X, we need $1 + P/10 > 2 + P/20$, so $P > 20$ ns.

The reason is that the average time is determined not only by the miss rate, but also by the hit time. A lower miss rate may come with a bigger cache size and thus a longer hit time. Therefore, when the punishment time (accessing main memory) is not that long, the smaller hit time might perform better.

Declaration of AI Use

I used an AI assistant on all problems in this homework, for grammar polishing, LaTeX typesetting, and checking the correctness of my arguments. All mathematical ideas and proofs are my own.